Deep Generative Models from Scratch: VAE and DDPM

1. Introduction

Deep generative models aim to learn the underlying distribution of a dataset and generate new samples that resemble the original data. Among the important families of generative models are Variational Autoencoders (VAEs) and Diffusion Models.

This project implements both a **Variational Autoencoder (VAE)** and a **Denoising Diffusion Probabilistic Model (DDPM)** from the ground up. The objective is not only to obtain generated images, but also to understand the fundamental differences between the two approaches, including their architectures, training objectives, generation procedures, computational requirements, and resulting image quality.

Both models were trained on the same public dataset, **CIFAR-10**, allowing their generative capabilities to be compared under the same general data domain. The models were evaluated using both quantitative metrics, specifically **Fréchet Inception Distance (FID)** and **Inception Score (IS)**, and qualitative inspection of generated samples.

The complete implementation is available in the project repository:

GenCV003: https://github.com/abdulrahman1238/GenCV003

---

2. Dataset

2.1 CIFAR-10

CIFAR-10 was selected as the common dataset for both experiments.

CIFAR-10 contains small RGB images belonging to 10 object categories. Each image has a spatial resolution of **32 × 32 pixels** and three color channels.

The dataset is particularly suitable for this assignment because:

- It is publicly available.
- It is widely used as a benchmark for generative modeling.
- Its relatively small image resolution makes training from scratch feasible.
- It contains multiple semantic categories, allowing image diversity and class coverage to be visually examined.

Both models operate directly on 32 × 32 RGB images.

The DDPM configuration explicitly defines CIFAR-10 as the dataset, with an image size of 32 and 3 input channels. The VAE configuration uses the same image dimensions.

---

3. Overall Methodology

The two models learn the data distribution in fundamentally different ways.

The VAE learns a continuous latent representation and reconstructs images from that latent space.

The DDPM instead learns how to reverse a gradual noise corruption process, starting generation from random Gaussian noise and progressively denoising it.

---

4. Variational Autoencoder

4.1 Overview

A Variational Autoencoder is a probabilistic generative model consisting primarily of an encoder and decoder.

Unlike a conventional autoencoder, which maps an image to a single deterministic latent vector, a VAE maps an input image to the parameters of a probability distribution, typically a Gaussian distribution.

The encoder therefore predicts:

- Mean: μ
- Log-variance: $\log(\sigma^2)$

A latent vector is then sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \varepsilon$$

where:

$$\varepsilon \sim N(0, I)$$

This allows the sampling operation to remain differentiable during training.

The decoder then maps the sampled latent vector back into image space.

The VAE formulation follows the variational inference approach introduced by Kingma and Welling.

---

5. VAE Architecture

The implemented VAE contains two main components:

Encoder

The encoder consists of convolutional blocks that progressively reduce the spatial resolution while increasing the number of feature channels.

The implementation dynamically builds convolutional layers using:

- 3 × 3 convolutions
- Stride = 2
- Batch normalization
- ReLU activation

The configured hidden dimensions are:

[64, 128, 256]

For a 32 × 32 image, the spatial dimensions are progressively reduced:

32 × 32
↓
16 × 16
↓
8 × 8
↓
4 × 4

The resulting representation is flattened and passed through two fully connected layers:

Encoder representation
|
+----> μ
|
+----> log(σ²)

The latent dimensionality is:

latent_dim = 128

5.1 Reparameterization

The model samples the latent variable using:

$z = \mu + \sigma \varepsilon$

where:

$\sigma = \exp(0.5 \times \text{logvar})$
$\varepsilon \sim N(0, I)$

This is important because directly sampling from a distribution would break the normal gradient-based optimization process.

The reparameterization trick separates the randomness from the learnable parameters, allowing gradients to flow through μ and σ.

---

6. VAE Decoder

The decoder performs the reverse transformation.

The 128-dimensional latent vector is first projected into a spatial feature representation.

The decoder then uses transposed convolution layers to progressively increase the spatial resolution:

4 × 4
↓
8 × 8
↓
16 × 16
↓
32 × 32

The final layer uses a sigmoid activation because the VAE operates on images normalized to the `[0, 1]` range.

Therefore:

Latent vector
↓
Fully connected layer
↓
4 × 4 feature map
↓
Transposed convolutions
↓
32 × 32 × 3 image
↓
Sigmoid

---

7. VAE Training Objective

The VAE is trained using two main components:

1. Reconstruction loss
2. KL divergence

The total loss can be expressed as:

$L = L_{reconstruction} + \beta L_{KL}$

The reconstruction term encourages the decoder to produce an image similar to the original input.

The KL divergence term regularizes the learned latent distribution so that it remains close to a standard Gaussian distribution.

In the implementation, binary cross-entropy is used as the reconstruction loss and the KL term is weighted by:

$\beta = 1e\text{-}4$

The relatively small KL weight allows the model to prioritize reconstruction quality while still providing latent-space regularization.

---

8. VAE Training Configuration

The main configuration used for the VAE was:

Parameter	Value
Dataset	CIFAR-10
Image size	32 × 32
Channels	3
Latent dimension	128
Hidden dimensions	[64, 128, 256]
Batch size	128
Learning rate	1×10^{-3}
Epochs	50
KL weight	1×10^{-4}
Output range	[0, 1]

The training implementation performs a forward pass, computes reconstruction and KL losses, performs backpropagation, and updates the model parameters using the optimizer.

Generated sample grids are also saved periodically during training.

---

9. Denoising Diffusion Probabilistic Model

9.1 Overview

A DDPM takes a fundamentally different approach from a VAE.

Instead of learning a direct mapping from an image to a compact latent representation, the diffusion model defines a sequence of increasingly noisy versions of the image.

The forward diffusion process gradually adds Gaussian noise:

$x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \ldots \rightarrow x\_T$

where:

- x_0 is the original image.
- x_T is approximately pure Gaussian noise.

The model is then trained to learn the reverse process:

x_T → x_(T-1) → ... → x_1 → x_0

Therefore, generation starts from random noise and progressively reconstructs an image.

This approach was introduced in the DDPM framework by Ho, Jain, and Abbeel.

---

10. DDPM Forward Diffusion

The forward process gradually destroys the information contained in the original image.

At each timestep, Gaussian noise is added according to a predefined variance schedule.

The implementation uses:

timesteps = 1000

and a cosine beta schedule.

The general formulation is:

q(x_t | x_{t-1}) = N(√(1 - β_t)x_{t-1}, β_t I)

Instead of applying all 1,000 transformations explicitly during training, the implementation can directly construct a noisy image x_t from x_0 using the cumulative noise schedule.

This allows different noise levels to be sampled efficiently during training.

---

11. DDPM U-Net Architecture

The denoising network is a convolutional U-Net.

The configuration uses:

channels = [64, 128, 256]

The U-Net contains:

- Downsampling blocks
- Residual blocks
- Time embeddings
- Skip connections
- Upsampling blocks
- Self-attention

The timestep is embedded and injected into the network so that the model knows how much noise is currently present in the image.

This is essential because the denoising task changes depending on the diffusion timestep.

---

12. Self-Attention

The implementation includes self-attention at the low spatial resolution.

The attention mechanism allows features at different spatial locations to interact with each other.

This can help the model capture global image structure rather than relying entirely on local convolutional operations.

The implementation specifically applies attention around the 8 × 8 resolution to balance global modeling capability and computational cost.

---

13. DDPM Training Objective

During training:

1. A clean image x_0 is selected.
2. A random timestep t is sampled.
3. Gaussian noise ε is generated.
4. The clean image is corrupted according to the diffusion schedule.
5. The model receives the noisy image and timestep.
6. The model predicts the noise that was added.
7. Mean squared error is calculated between the predicted and actual noise.

The simplified objective is:

$$L = E[|| \varepsilon - \varepsilon_\theta(x_t, t) ||^2]$$

where:

- ε is the actual Gaussian noise.
- ε_θ is the noise predicted by the neural network.

Learning this noise prediction function allows the model to approximate the reverse diffusion process.

---

14. DDPM Sampling

Generation begins with a randomly sampled Gaussian noise image:

$x_T \sim N(0, I)$

The model then repeatedly predicts the noise and calculates the previous state.

The implementation performs:

x_T
↓
x_999
↓
x_998
↓
...
↓
x_1
↓
x_0

for 1,000 diffusion steps.

Intermediate states can also be saved to visualize the complete denoising trajectory.

This provides a useful qualitative demonstration of how the diffusion model transforms random noise into a structured image.

---

15. DDPM Training Configuration

The main DDPM configuration was:

Parameter	Value
Dataset	CIFAR-10
Image size	32 × 32
Channels	3
Batch size	64
Learning rate	2×10^{-4}
Epochs	100
Diffusion timesteps	1000
Beta schedule	Cosine
U-Net channels	[64, 128, 256]
Time embedding dimension	128
Residual blocks	2
Attention	Enabled
Dropout	0.1
Seed	42

Samples and denoising trajectories are generated periodically during training.

---

16. Quantitative Evaluation

Two metrics were used to evaluate the generated images:

1. Fréchet Inception Distance (FID)
2. Inception Score (IS)

Both metrics are widely used for evaluating image generative models.

The project implements a common evaluation interface so that both models can be sampled through the same evaluation pipeline.

The evaluation implementation generates **500 images** for calculating the reported metrics.

---

17. Experimental Results

The final quantitative results were:

Model	FID ↓	Inception Score ↑
VAE	**180.29**	2.96 ± 0.34
DDPM	338.61	2.24 ± 0.32

The VAE obtained the better result on both metrics in this experiment.

Specifically:

- VAE FID: 180.29
- DDPM FID: **338.61**
- VAE IS: **2.96 ± 0.34**
- DDPM IS: **2.24 ± 0.32**

The VAE therefore produced generated samples that were quantitatively closer to the real-image distribution than the DDPM under the experimental setup used in this project.

---

18. Honest Interpretation of the Results

The obtained FID values are **high**, and the Inception Scores are **low** compared with strong published generative-model results on CIFAR-10.

Therefore, the results are bad, it isn't as expected.

For context, the original DDPM paper reported substantially stronger CIFAR-10 results, including an FID of 3.17 and an Inception Score of 9.46 for its best configuration.

---

19. Fundamental Difference Between VAE and DDPM

The most important difference is **how each model represents and learns the data distribution**.

A VAE attempts to learn:

Image → Latent Distribution → Image

The model compresses an image into a probabilistic latent representation and then reconstructs it.

The DDPM instead learns:

Image → Noisy Image → ... → Pure Noise

during training and learns the reverse transformation:

Pure Noise → ... → Image

during generation.

Therefore:

> **VAE generation is latent-space decoding, whereas DDPM generation is iterative denoising.**

This difference has major practical consequences.

The VAE has a relatively inexpensive generation process because a single latent vector can be decoded directly into an image.

The DDPM performs many sequential denoising operations, making generation significantly more computationally expensive.

On the other hand, diffusion models have demonstrated an ability to produce highly realistic and detailed images when adequately trained and scaled.

---

20. Conclusion

This project implemented two fundamentally different deep generative modeling approaches from scratch: a Variational Autoencoder and a Denoising Diffusion Probabilistic Model.

The VAE learns a probabilistic latent representation and generates images by decoding samples from this latent space. Its main advantages are conceptual simplicity, efficient sampling, and an explicit latent representation.

The DDPM learns a reverse diffusion process that transforms Gaussian noise into structured images through many iterative denoising steps. This approach is computationally more expensive, but diffusion models have demonstrated strong image-generation capabilities when sufficiently scaled and trained.

In the conducted experiment, the VAE achieved:

FID:180.29
Inception Score: **2.96 ± 0.34**

while the DDPM achieved:

FID: **338.61**
Inception Score: :**2.24 ± 0.32**

Therefore, the VAE performed better according to both reported quantitative metrics in this particular experiment.

However, the results should be interpreted honestly: **both FID values are high and both Inception Scores are relatively low**, indicating that the models did not achieve strong generative quality under the selected training setup.

This should not be interpreted as evidence that VAEs are inherently superior to DDPMs. Instead, it demonstrates the effect of model capacity, training duration, optimization, sampling strategy, and computational budget on generative performance.

The most important outcome of the project is therefore not simply which model obtained the lower FID. The experiment demonstrates the fundamental difference between two important generative modeling paradigms:

> **VAEs learn to generate through a structured probabilistic latent space, while DDPMs learn generation as the reversal of a gradual noise-diffusion process.**

The VAE offers a faster and simpler generation mechanism, while DDPMs provide a more powerful but computationally expensive framework for modeling complex image distributions.

---

21. References

1. Kingma, D. P., & Welling, M. — *Auto-Encoding Variational Bayes*, 2013.
2. Ho, J., Jain, A., & Abbeel, P. — *Denoising Diffusion Probabilistic Models*, NeurIPS 2020.
3. Salimans, T., et al. — *Improved Techniques for Training GANs*, NeurIPS 2016. The work introduced the Inception Score as part of its evaluation methodology.
4. Heusel, M., et al. — *GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium*, NeurIPS 2017. This work introduced the Fréchet Inception Distance (FID).

5. CIFAR-10 Dataset — Krizhevsky, A. — *Learning Multiple Layers of Features from Tiny Images*.